Отчет
о выполнении лабораторной работы № 3
Сети Элмана, Хопфилда и Хэмминга
Вариант 13

Выполнил студент группы ЗФИбд-01-24: Князев А. М.

Проверил: Леонов С. С.

Москва, 2026

Цель работы

исследование свойств сетей Элмана, Хопфилда и Хэмминга, а также их
применение для распознавания динамических и статических образов.

Основные этапы работы

1. Построить и обучить сеть Элмана для распознавания динамического
образа.

2. Построить сеть Хопфилда для хранения и восстановления статическ
образов.

3. Реализовать сеть Хэмминга для классификации зашумленных образо

Ход выполнения и результаты

Таблица 1 — Параметры варианта

Параметр | Значение

g(k) | sin(2k^2 - 6k + 3), k in [-0.02, 2.36]

R | [2, 5, 6]

Образы | цифры 0, 1, 4 размером 12x10

Устройство при проверке | cuda

Таблица 2 — Результаты

Этап | Результат

Элман, обучение | loss = 0.295058, правильно 751/821

Элман, измененное R=[2,4,6] | правильно 710/780

Хопфилд | восстановлены цифры 0, 1, 4

Хэмминг | выбраны цифры 0, 1, 4

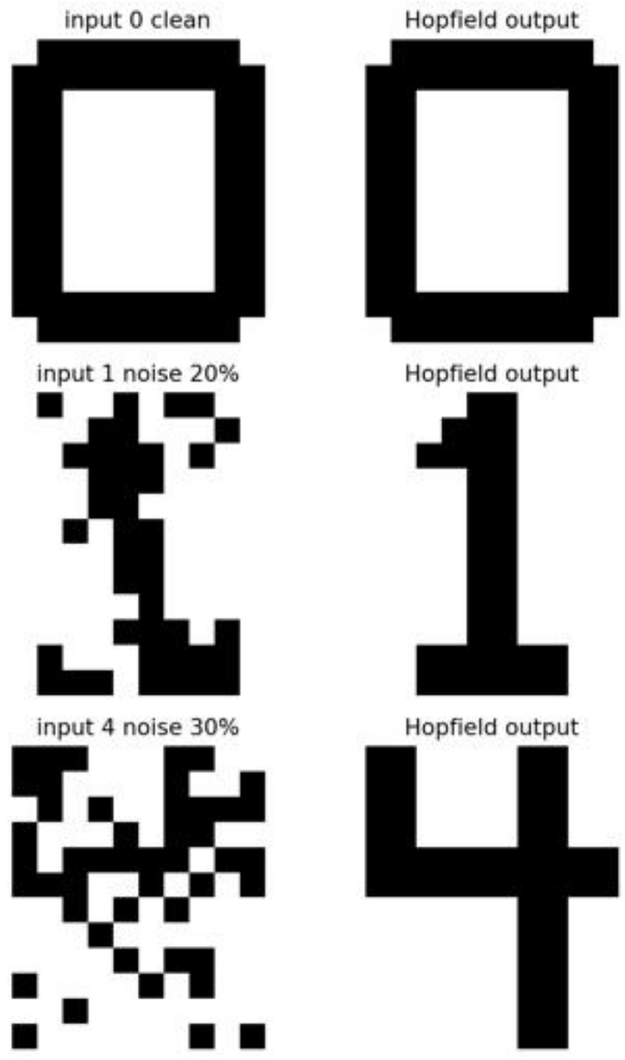


Рис. 1 — Входные и восстановленные образы сети Хопфилда

Вывод

Сеть Элмана распознает участки последовательности, соответствующи
заданному динамическому образу. Сети Хопфилда и Хэмминга коррект
распознают выбранные статические образы даже после внесения шума
30%.

Фрагменты кода из main.ipynb

```
from __future__ import annotations


import math
import random
from dataclasses import dataclass



import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEED = 13
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)


def variant_signal(k: np.ndarray) -> np.ndarray:
    return np.sin(2 * k**2 - 6 * k + 3)


def make_elman_sequence(repeats: list[int]) -> tuple[np.ndarray, np.ndarray]:
    step = 0.025
    k1 = np.arange(0.0, 1.0 + step / 2, step)
    k2 = np.arange(-0.02, 2.36 + step / 2, step)

    p1 = np.sin(4 * math.pi * k1)
    p2 = variant_signal(k2)
    t1 = -np.ones_like(p1)
    t2 = np.ones_like(p2)

    parts_x: list[np.ndarray] = []
    parts_y: list[np.ndarray] = []
    for count in repeats:
        parts_x.extend([p1] * count)
        parts_x.append(p2)
        parts_y.extend([t1] * count)
        parts_y.append(t2)
    return np.concatenate(parts_x), np.concatenate(parts_y)
```

```
class ElmanNet(nn.Module):
    def __init__(self, hidden_size: int = 8):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=hidden_size,
            nonlinearity="tanh",
            batch_first=True,
        )
        self.output = nn.Linear(hidden_size, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        hidden, _ = self.rnn(x)
        return torch.tanh(self.output(hidden))


@dataclass
class ElmanResult:
    loss: float
    correct: int
    total: int


def train_elman(x_np: np.ndarray, y_np: np.ndarray) -> tuple[ElmanNet, ElmanResult]:
    model = ElmanNet(hidden_size=8).to(DEVICE)
    x = torch.tensor(x_np, dtype=torch.float32, device=DEVICE).view(1, -1, 1)
    y = torch.tensor(y_np, dtype=torch.float32, device=DEVICE).view(1, -1, 1)

    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    criterion = nn.MSELoss()
    best_loss = float("inf")

    for _ in range(1000):
        optimizer.zero_grad()
        pred = model(x)
        loss = criterion(pred, y)
        loss.backward()
        optimizer.step()
        best_loss = min(best_loss, float(loss.item()))
        if loss.item() < 1e-5:
            break

with torch.no_grad():
    pred = model(x).squeeze().detach().cpu().numpy()
```

```
    classes = np.where(pred >= 0, 1, -1)
    correct = int((classes == y_np).sum())
    return model, ElmanResult(loss=best_loss, correct=correct, total=len(y_np))


DIGITS = {
    0: [
        "0111111110",
        "1111111111",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1111111111",
        "0111111110",
    ],
    1: [
        "0000110000",
        "0001110000",
        "0011110000",


def digit_to_pattern(digit: int) -> np.ndarray:
    image = np.array([[int(ch) for ch in row] for row in DIGITS[digit]], dtype=np.float32)
    return np.where(image > 0, 1.0, -1.0)


def add_noise(pattern: np.ndarray, amount: float, seed: int) -> np.ndarray:
    rng = np.random.default_rng(seed)
    noisy = pattern.copy()
    mask = rng.random(pattern.shape) < amount
    noisy[mask] *= -1
    return noisy


def train_hopfield(patterns: list[np.ndarray]) -> np.ndarray:
    vectors = [p.reshape(-1) for p in patterns]
    size = vectors[0].size
    weights = np.zeros((size, size), dtype=np.float32)
    for vector in vectors:
        weights += np.outer(vector, vector)
```

```
    np.fill_diagonal(weights, 0.0)
    return weights / size


def run_hopfield(weights: np.ndarray, pattern: np.ndarray, iterations: int = 600) -> np.ndarray:
    state = pattern.reshape(-1).copy()
    for _ in range(iterations):
        updated = np.sign(weights @ state)
        updated[updated == 0] = state[updated == 0]
        if np.array_equal(updated, state):
            break
        state = updated
    return state.reshape(pattern.shape)
```